Gramáticas para lenguajes no libres de contexto

Modelos de Computación · Relación 1 bis

Ejercicios *extremadamente difíciles*

Ejercicios

Ninguno de estos lenguajes es libre de contexto, así que necesitaremos **gramáticas de tipo 0 / 1**.

a) $\{ ww \mid w \in \{0,1\}^* \}$

b) $\{ a^{n^2} \mid n \geq 0 \}$

c) $\{ a^p \mid p \text{ primo} \}$

d) $\{ a^n b^m \mid n \leq m^2 \}$

Antes de empezar: Pensar en qué necesitamos

Antes de empezar: Pensar en qué necesitamos

- **Marcadores**: variables que "recuerdan" algo pendiente de hacer. p. ej. A = "aquí falta copiar un 0"

Antes de empezar: Pensar en qué necesitamos

- **Marcadores**: variables que "recuerdan" algo pendiente de hacer. p. ej. A = "aquí falta copiar un 0"
- **Desplazamientos**: una variable se mueve intercambiándose con su vecina. $A0 \to 0A$ hace que A avance hacia la derecha

Antes de empezar: Pensar en qué necesitamos

- **Marcadores**: variables que "recuerdan" algo pendiente de hacer. p. ej. A = "aquí falta copiar un 0"
- **Desplazamientos**: una variable se mueve intercambiándose con su vecina. $A0 \to 0A$ hace que A avance hacia la derecha
- **Cursores**: una variable que recorre la cadena y va transformando lo que encuentra.

Antes de empezar: Pensar en qué necesitamos

- **Marcadores**: variables que "recuerdan" algo pendiente de hacer. p. ej. A = "aquí falta copiar un 0"
- **Desplazamientos**: una variable se mueve intercambiándose con su vecina. $A0 \to 0A$ hace que A avance hacia la derecha
- **Cursores**: una variable que recorre la cadena y va transformando lo que encuentra.
- **Bordes**: variables que marcan el principio o el final para saber cuándo parar.

Antes de empezar: Pensar en qué necesitamos

- **Marcadores**: variables que "recuerdan" algo pendiente de hacer. p. ej. A = "aquí falta copiar un 0"
- **Desplazamientos**: una variable se mueve intercambiándose con su vecina. $A0 \to 0A$ hace que A avance hacia la derecha
- **Cursores**: una variable que recorre la cadena y va transformando lo que encuentra.
- **Bordes**: variables que marcan el principio o el final para saber cuándo parar.
- **Limpieza**: al final hay que hacer desaparecer todas las variables auxiliares.

Antes de empezar: Pensar en qué necesitamos

- **Marcadores**: variables que "recuerdan" algo pendiente de hacer. p. ej. A = "aquí falta copiar un 0"
- **Desplazamientos**: una variable se mueve intercambiándose con su vecina. $A0 \to 0A$ hace que A avance hacia la derecha
- **Cursores**: una variable que recorre la cadena y va transformando lo que encuentra.
- **Bordes**: variables que marcan el principio o el final para saber cuándo parar.
- **Limpieza**: al final hay que hacer desaparecer todas las variables auxiliares.

> 💡 La gramática no "elige bien" el orden de aplicación: hay que diseñarla para que **cualquier camino equivocado se quede bloqueado** (con variables que ya no se pueden eliminar).

a) $\{ ww \mid w \in \{0,1\}^* \}$

Una cadena repetida dos veces

¿Qué palabras hay en el lenguaje?

Palabras de la forma $w\,w$: la primera mitad es **idéntica** a la segunda.

✅ Pertenecen

palabra	w
ε	ε
00	0
0101	01
$110\,110$	110

¿Qué palabras hay en el lenguaje?

Palabras de la forma $w\,w$: la primera mitad es **idéntica** a la segunda.

✅ Pertenecen

palabra	w
ε	ε
00	0
0101	01
$110\,110$	110

❌ No pertenecen

palabra	motivo
010	longitud impar
0011	$00 \neq 11$
$01\,10$	es ww^{-1}

La idea

Escribimos w y por cada símbolo dejamos una **nota** de lo que hay que copiar. Las notas viajan al final y allí se transforman.

1. Generar *w* dejando marcadores



La idea

Escribimos w y por cada símbolo dejamos una **nota** de lo que hay que copiar. Las notas viajan al final y allí se transforman.

1. Generar w dejando marcadores 

2. Los marcadores van a la derecha 

La idea

Escribimos w y por cada símbolo dejamos una **nota** de lo que hay que copiar. Las notas viajan al final y allí se transforman.

1. Generar *w* dejando marcadores 

2. Los marcadores van a la derecha 

3. El cursor C los recorre hacia la izquierda 

La idea

Escribimos w y por cada símbolo dejamos una **nota** de lo que hay que copiar. Las notas viajan al final y allí se transforman.

1. Generar *w* dejando marcadores 

2. Los marcadores van a la derecha 

3. El cursor C los recorre hacia la izquierda 

4. D convierte los marcadores en la copia 

La idea

Escribimos w y por cada símbolo dejamos una **nota** de lo que hay que copiar. Las notas viajan al final y allí se transforman.

1. Generar *w* dejando marcadores



2. Los marcadores van a la derecha



3. El cursor C los recorre hacia la izquierda



4. D convierte los marcadores en la copia



Ejemplo con $w = 010$. Los terminales de w nunca se mueven: sólo se mueven las variables.

Símbolos auxiliares

símbolo	significado
S	símbolo inicial: va generando w
A	"tengo pendiente copiar un 0"
B	"tengo pendiente copiar un 1"
C	marca el final de w y luego recorre los marcadores hacia la izquierda
A′ B′	marcador ya **visitado** por C: listo para escribirse
D	convierte $A' \to 0$ y $B' \to 1$ de izquierda a derecha

Paso 1 · Generar w y guardar una copia

$$S \;\to\; 0AS \;\mid\; 1BS \;\mid\; C \;\mid\; \varepsilon$$

Paso 1 · Generar w y guardar una copia

$$S \rightarrow 0AS \mid 1BS \mid C \mid \varepsilon$$

- Cada vez que escribimos un terminal, dejamos **su marcador justo detrás.**

Paso 1 · Generar w y guardar una copia

$$S \rightarrow 0AS \mid 1BS \mid C \mid \varepsilon$$

- Cada vez que escribimos un terminal, dejamos **su marcador justo detrás**.
- Cuando decidimos que w ha terminado, ponemos C (marca de fin).

Paso 1 · Generar w y guardar una copia

$$S \;\to\; 0AS \;\mid\; 1BS \;\mid\; C \;\mid\; \varepsilon$$

- Cada vez que escribimos un terminal, dejamos **su marcador justo detrás**.
- Cuando decidimos que w ha terminado, ponemos C (marca de fin).
- $S \to \varepsilon$ sólo sirve para la palabra vacía ($w = \varepsilon$).

Paso 1 · Generar w y guardar una copia

$$S \to 0AS \mid 1BS \mid C \mid \varepsilon$$

- Cada vez que escribimos un terminal, dejamos **su marcador justo detrás**.
- Cuando decidimos que w ha terminado, ponemos C (marca de fin).
- $S \to \varepsilon$ sólo sirve para la palabra vacía ($w = \varepsilon$).

Para $w = 01$: S ⇒ 0 A S ⇒ 0 A 1 B S ⇒ 0 A 1 B C

Paso 1 · Generar w y guardar una copia

$$S \rightarrow 0AS \mid 1BS \mid C \mid \varepsilon$$

- Cada vez que escribimos un terminal, dejamos **su marcador justo detrás**.
- Cuando decidimos que w ha terminado, ponemos C (marca de fin).
- $S \rightarrow \varepsilon$ sólo sirve para la palabra vacía ($w = \varepsilon$).

Para $w = 01$: `S` ⇒ `0` `A` `S` ⇒ `0` `A` `1` `B` `S` ⇒ `0` `A` `1` `B` `C`

Ya tenemos **la primera copia** (los terminales) y **la segunda codificada** (los marcadores, en el mismo orden). Pero están mezcladas.

Paso 2 · Los marcadores van hacia la derecha

$$A0 \to 0A \qquad A1 \to 1A \qquad\qquad B0 \to 0B \qquad B1 \to 1B$$

Paso 2 · Los marcadores van hacia la derecha

$$A0 \to 0A \qquad A1 \to 1A \qquad\qquad B0 \to 0B \qquad B1 \to 1B$$

- Un marcador puede **saltar un terminal** hacia la derecha.

Paso 2 · Los marcadores van hacia la derecha

$$A0 \to 0A \qquad A1 \to 1A \qquad\qquad B0 \to 0B \qquad B1 \to 1B$$

- Un marcador puede **saltar un terminal** hacia la derecha.
- **No hay** reglas $AB \to BA$ ni $BA \to AB$: los marcadores **nunca se adelantan entre sí**. → conservan el orden de w.

Paso 2 · Los marcadores van hacia la derecha

$$A0 \to 0A \qquad A1 \to 1A \qquad\qquad B0 \to 0B \qquad B1 \to 1B$$

- Un marcador puede **saltar un terminal** hacia la derecha.
- **No hay** reglas $AB \to BA$ ni $BA \to AB$: los marcadores **nunca se adelantan entre sí**. → conservan el orden de w.
- Los terminales no cambian de orden: siguen formando w.

Paso 2 · Los marcadores van hacia la derecha

$$A0 \to 0A \qquad A1 \to 1A \qquad\qquad B0 \to 0B \qquad B1 \to 1B$$

- Un marcador puede **saltar un terminal** hacia la derecha.
- **No hay** reglas $AB \to BA$ ni $BA \to AB$: los marcadores **nunca se adelantan entre sí**. → conservan el orden de w.
- Los terminales no cambian de orden: siguen formando w.



(regla `A1 → 1A`)

Paso 2 · Los marcadores van hacia la derecha

$$A0 \to 0A \qquad A1 \to 1A \qquad\qquad B0 \to 0B \qquad B1 \to 1B$$

- Un marcador puede **saltar un terminal** hacia la derecha.
- **No hay** reglas $AB \to BA$ ni $BA \to AB$: los marcadores **nunca se adelantan entre sí**. → conservan el orden de w.
- Los terminales no cambian de orden: siguen formando w.

`0` `A` `1` `B` `C` ⇒ `0` `1` `A` `B` `C` (regla `A1 → 1A`)

Objetivo: dejar `0` `1` terminales a la izquierda y `A` `B` marcadores pegados a `C`.

Paso 3 · El cursor C recorre los marcadores

$$AC \to CA' \qquad\qquad BC \to CB'$$

Paso 3 · El cursor C recorre los marcadores

$$AC \to CA' \qquad\qquad BC \to CB'$$

- C avanza hacia la **izquierda** cruzando marcadores.

Paso 3 · El cursor C recorre los marcadores

$$AC \to CA' \qquad\qquad BC \to CB'$$

- C avanza hacia la **izquierda** cruzando marcadores.
- Al cruzarlos los marca con $'$ ("ya visitado"): así **no pueden volver a moverse** (no hay reglas para A' salvo la de escribir).

Paso 3 · El cursor C recorre los marcadores

$$AC \to CA' \qquad\qquad BC \to CB'$$

- C avanza hacia la **izquierda** cruzando marcadores.
- Al cruzarlos los marca con $'$ ("ya visitado"): así **no pueden volver a moverse** (no hay reglas para A' salvo la de escribir).
- C **sólo** sabe cruzar marcadores, nunca terminales.

Paso 3 · El cursor C recorre los marcadores

$$AC \to CA' \qquad\qquad BC \to CB'$$

- C avanza hacia la **izquierda** cruzando marcadores.
- Al cruzarlos los marca con $'$ ("ya visitado"): así **no pueden volver a moverse** (no hay reglas para A' salvo la de escribir).
- C **sólo** sabe cruzar marcadores, nunca terminales.



Paso 4 · Escribir la segunda copia

$$0C \to 0D \qquad 1C \to 1D \qquad DA' \to 0D \qquad DB' \to 1D \qquad D \to \varepsilon$$

Paso 4 · Escribir la segunda copia

$$0C \to 0D \qquad 1C \to 1D \qquad DA' \to 0D \qquad DB' \to 1D \qquad D \to \varepsilon$$

- Cuando C choca con un terminal ya ha cruzado todos los marcadores: se transforma en el escritor D.

Paso 4 · Escribir la segunda copia

$$0C \to 0D \qquad 1C \to 1D \qquad DA' \to 0D \qquad DB' \to 1D \qquad D \to \varepsilon$$

- Cuando C choca con un terminal ya ha cruzado todos los marcadores: se transforma en el escritor D.
- D avanza hacia la **derecha** y convierte cada marcador visitado en su terminal.

Paso 4 · Escribir la segunda copia

$$0C \to 0D \qquad 1C \to 1D \qquad DA' \to 0D \qquad DB' \to 1D \qquad D \to \varepsilon$$

- Cuando C choca con un terminal ya ha cruzado todos los marcadores: se transforma en el escritor D.
- D avanza hacia la **derecha** y convierte cada marcador visitado en su terminal.
- Al final D desaparece.

Paso 4 · Escribir la segunda copia

$$0C \to 0D \qquad 1C \to 1D \qquad DA' \to 0D \qquad DB' \to 1D \qquad D \to \varepsilon$$

- Cuando C choca con un terminal ya ha cruzado todos los marcadores: se transforma en el escritor D.
- D avanza hacia la **derecha** y convierte cada marcador visitado en su terminal.
- Al final D desaparece.



La gramática completa

$G = (V, \{0,1\}, P, S)$ con $V = \{S, A, B, A', B', C, D\}$ y P:

① generar

$$S \to 0AS$$
$$S \to 1BS$$
$$S \to C$$
$$S \to \varepsilon$$

② marcadores →

$$A0 \to 0A$$
$$A1 \to 1A$$
$$B0 \to 0B$$
$$B1 \to 1B$$

③ cursor ←

$$AC \to CA'$$
$$BC \to CB'$$

④ escribir →

$$0C \to 0D$$
$$1C \to 1D$$
$$DA' \to 0D$$
$$DB' \to 1D$$
$$D \to \varepsilon$$

La gramática completa

$G = (V, \{0,1\}, P, S)$ con $V = \{S, A, B, A', B', C, D\}$ y P:

① generar

$$S \to 0AS$$
$$S \to 1BS$$
$$S \to C$$
$$S \to \varepsilon$$

② marcadores →

$$A0 \to 0A$$
$$A1 \to 1A$$
$$B0 \to 0B$$
$$B1 \to 1B$$

③ cursor ←

$$AC \to CA'$$
$$BC \to CB'$$

④ escribir →

$$0C \to 0D$$
$$1C \to 1D$$
$$DA' \to 0D$$
$$DB' \to 1D$$
$$D \to \varepsilon$$

Las reglas se pueden aplicar **en cualquier orden**. Lo que garantiza que sólo salgan palabras ww es que los caminos "malos" se bloquean.

Derivación completa: $w = 01$

S

PASO 0 / 10

S

símbolo inicial

Derivación completa: $w = 01$

PASO 1 / 10



regla `S → 0AS`

> Escribimos 0 y dejamos su marcador A.

S

⇒ 0 A S S → 0AS

Derivación completa: $w = 01$

PASO 2 / 10



regla S → 1BS

> Escribimos 1 y dejamos su marcador B.

S

⇒ 0 A S S → 0AS

⇒ 0 A 1 B S S → 1BS

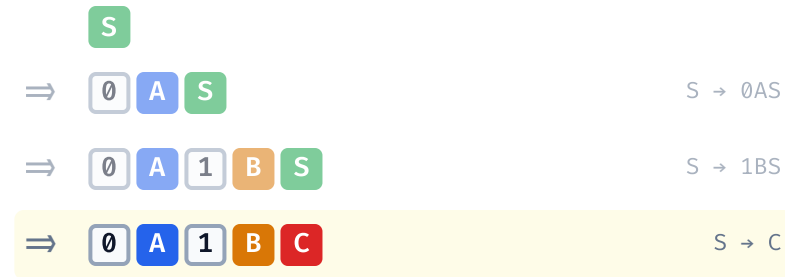
Derivación completa: $w = 01$

PASO 3 / 10



regla S → C

> w ha terminado: C marca el final.



Derivación completa: $w = 01$

PASO 4 / 10



regla `A1 → 1A`

> A salta el 1. Ya están los marcadores juntos.

S

⇒ 0 A S — S → 0AS

⇒ 0 A 1 B S — S → 1BS

⇒ 0 A 1 B C — S → C

⇒ 0 1 A B C — A1 → 1A

Derivación completa: $w = 01$

PASO 5 / 10



regla `BC → CB′`

> C cruza B y lo marca como visitado.



Derivación completa: $w = 01$

PASO 6 / 10



regla AC → CA’

> C cruza A. Ya no quedan marcadores sin visitar.

S

⇒ 0 A S — S → 0AS

⇒ 0 A 1 B S — S → 1BS

⇒ 0 A 1 B C — S → C

⇒ 0 1 A B C — A1 → 1A

⇒ 0 1 A C B’ — BC → CB’

⇒ 0 1 C A’ B’ — AC → CA’

Derivación completa: $w = 01$

PASO 7 / 10

0 1 D A′ B′

regla `1C → 1D`

> C toca un terminal: se convierte en el escritor D.

S

⇒ 0 A S — S → 0AS

⇒ 0 A 1 B S — S → 1BS

⇒ 0 A 1 B C — S → C

⇒ 0 1 A B C — A1 → 1A

⇒ 0 1 A C B′ — BC → CB′

⇒ 0 1 C A′ B′ — AC → CA′

⇒ 0 1 D A′ B′ — 1C → 1D

Derivación completa: $w = 01$

PASO 8 / 10

0 1 0 D B′

regla `DA′ → 0D`

> A' se convierte en 0.

S
⇒ 0 A S — S → 0AS
⇒ 0 A 1 B S — S → 1BS
⇒ 0 A 1 B C — S → C
⇒ 0 1 A B C — A1 → 1A
⇒ 0 1 A C B′ — BC → CB′
⇒ 0 1 C A′ B′ — AC → CA′
⇒ 0 1 D A′ B′ — 1C → 1D
⇒ 0 1 0 D B′ — DA′ → 0D

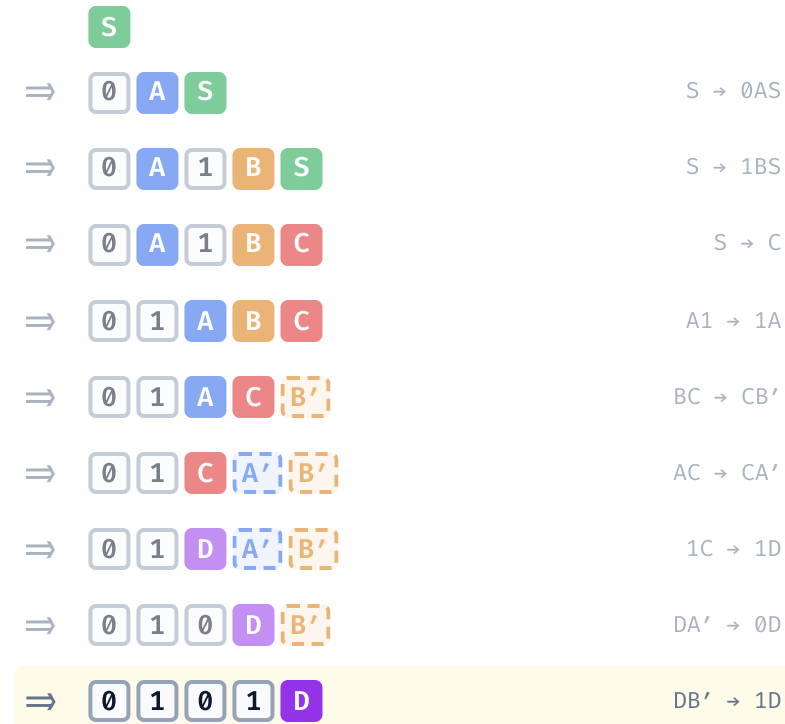
Derivación completa: $w = 01$

PASO 9 / 10

0 1 0 1 D

regla `DB′ → 1D`

> B' se convierte en 1.



Derivación completa: $w = 01$

PASO 10 / 10

0 1 0 1

regla `D → ε`

> ¡Palabra terminal! 0101 = (01)(01) ✅

⋮ 1 paso anterior

⇒ 0 A S — S → 0AS

⇒ 0 A 1 B S — S → 1BS

⇒ 0 A 1 B C — S → C

⇒ 0 1 A B C — A1 → 1A

⇒ 0 1 A C B′ — BC → CB′

⇒ 0 1 C A′ B′ — AC → CA′

⇒ 0 1 D A′ B′ — 1C → 1D

⇒ 0 1 0 D B′ — DA′ → 0D

⇒ 0 1 0 1 D — DB′ → 1D

⇒ **0 1 0 1** — D → ε

Otro ejemplo: $w = 101$

PASO 0 / 15

S

símbolo inicial

S

Otro ejemplo: $w = 101$

PASO 1 / 15

1 B S

regla S → 1BS

S

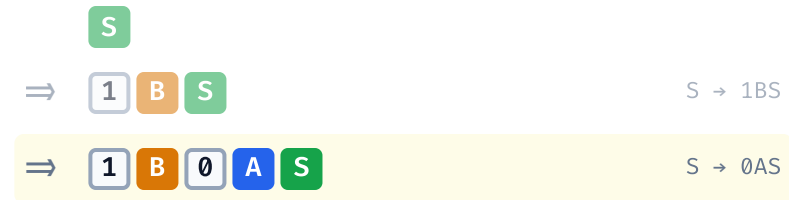
⇒ 1 B S S → 1BS

Otro ejemplo: $w = 101$

PASO 2 / 15



regla `S → 0AS`



Otro ejemplo: $w = 101$

PASO 3 / 15

1 B 0 A 1 B S

regla S → 1BS

S

⇒ 1 B S S → 1BS

⇒ 1 B 0 A S S → 0AS

⇒ 1 B 0 A 1 B S S → 1BS

Otro ejemplo: $w = 101$

PASO 4 / 15



regla S → C

> Primera copia escrita; marcadores B A B mezclados.

S

⇒ 1 B S — S → 1BS

⇒ 1 B 0 A S — S → 0AS

⇒ 1 B 0 A 1 B S — S → 1BS

⇒ 1 B 0 A 1 B C — S → C

Otro ejemplo: $w = 101$

PASO 5 / 15



regla `B0 → 0B`

S

⇒ 1 B S — S → 1BS

⇒ 1 B 0 A S — S → 0AS

⇒ 1 B 0 A 1 B S — S → 1BS

⇒ 1 B 0 A 1 B C — S → C

⇒ 1 0 B A 1 B C — B0 → 0B

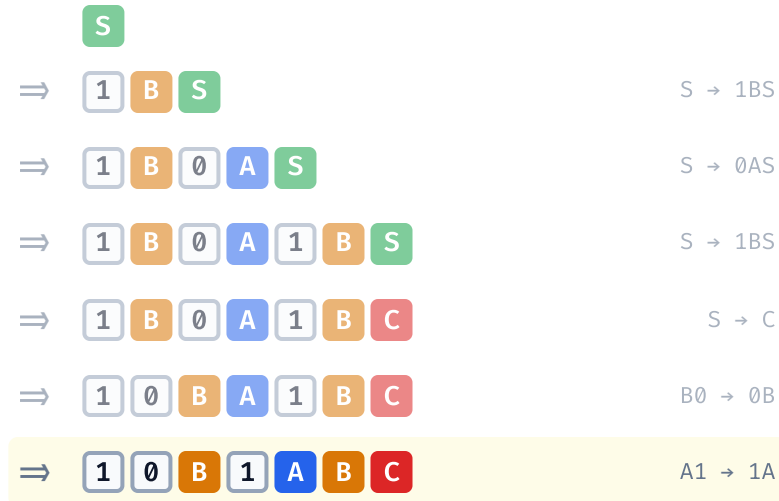
Otro ejemplo: $w = 101$

PASO 6 / 15



regla `A1 → 1A`

> B no puede adelantar a A: primero tiene que moverse A.



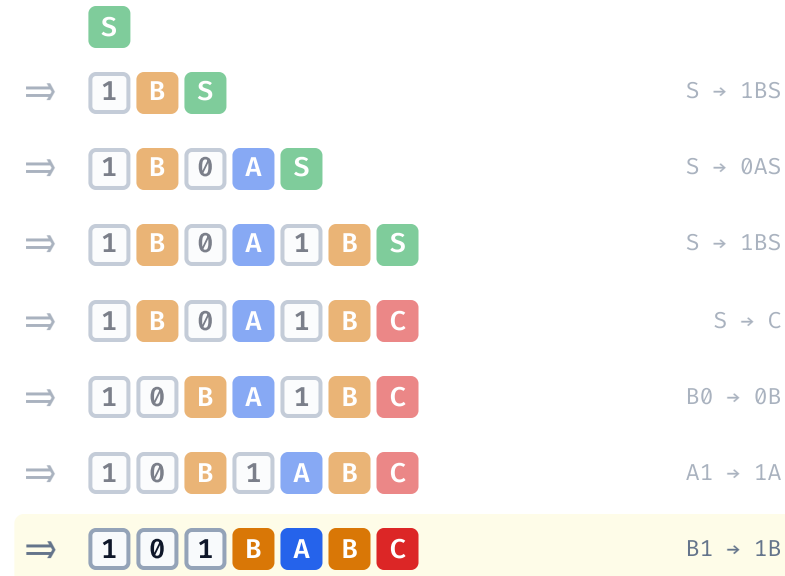
Otro ejemplo: $w = 101$

PASO 7 / 15



regla `B1 → 1B`

> Marcadores juntos y en el orden de w: B A B.



Otro ejemplo: $w = 101$

PASO 8 / 15



regla `BC → CB’`

S

⇒ 1 B S — S → 1BS

⇒ 1 B 0 A S — S → 0AS

⇒ 1 B 0 A 1 B S — S → 1BS

⇒ 1 B 0 A 1 B C — S → C

⇒ 1 0 B A 1 B C — B0 → 0B

⇒ 1 0 B 1 A B C — A1 → 1A

⇒ 1 0 1 B A B C — B1 → 1B

⇒ 1 0 1 B A C B’ — BC → CB’

Otro ejemplo: $w = 101$

PASO 9 / 15



regla AC → CA’

S

⇒ 1 B S — S → 1BS

⇒ 1 B 0 A S — S → 0AS

⇒ 1 B 0 A 1 B S — S → 1BS

⇒ 1 B 0 A 1 B C — S → C

⇒ 1 0 B A 1 B C — B0 → 0B

⇒ 1 0 B 1 A B C — A1 → 1A

⇒ 1 0 1 B A B C — B1 → 1B

⇒ 1 0 1 B A C B’ — BC → CB’

⇒ 1 0 1 B C A’ B’ — AC → CA’

Otro ejemplo: $w = 101$

PASO 10 / 15



regla BC → CB′

⋮ 1 paso anterior

⇒ 1 B S — S → 1BS

⇒ 1 B 0 A S — S → 0AS

⇒ 1 B 0 A 1 B S — S → 1BS

⇒ 1 B 0 A 1 B C — S → C

⇒ 1 0 B A 1 B C — B0 → 0B

⇒ 1 0 B 1 A B C — A1 → 1A

⇒ 1 0 1 B A B C — B1 → 1B

⇒ 1 0 1 B A C B′ — BC → CB′

⇒ 1 0 1 B C A′ B′ — AC → CA′

⇒ 1 0 1 C B′ A′ B′ — BC → CB′

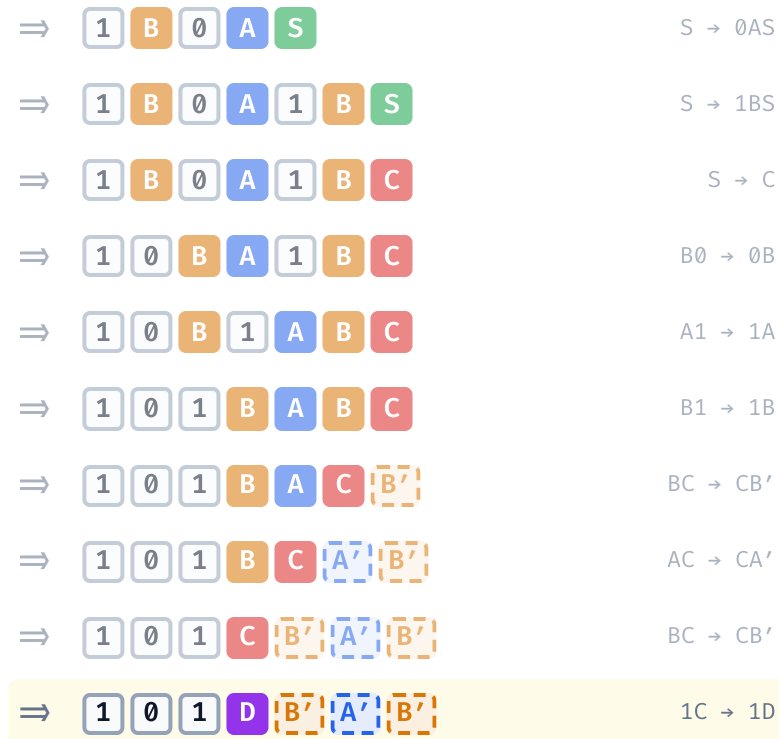
Otro ejemplo: $w = 101$

PASO 11 / 15



regla 1C → 1D

⋮ 2 pasos anteriores



Otro ejemplo: $w = 101$

PASO 12 / 15



regla `DB′ → 1D`

⋮ 3 pasos anteriores

⇒	1 B 0 A 1 B S	S → 1BS
⇒	1 B 0 A 1 B C	S → C
⇒	1 0 B A 1 B C	B0 → 0B
⇒	1 0 B 1 A B C	A1 → 1A
⇒	1 0 1 B A B C	B1 → 1B
⇒	1 0 1 B A C B′	BC → CB′
⇒	1 0 1 B C A′ B′	AC → CA′
⇒	1 0 1 C B′ A′ B′	BC → CB′
⇒	1 0 1 D B′ A′ B′	1C → 1D
⇒	1 0 1 1 D A′ B′	DB′ → 1D

Otro ejemplo: $w = 101$

PASO 13 / 15



regla `DA′ → 0D`

⋮ 4 pasos anteriores

⇒	1 B 0 A 1 B C	S → C
⇒	1 0 B A 1 B C	B0 → 0B
⇒	1 0 B 1 A B C	A1 → 1A
⇒	1 0 1 B A B C	B1 → 1B
⇒	1 0 1 B A C B′	BC → CB′
⇒	1 0 1 B C A′ B′	AC → CA′
⇒	1 0 1 C B′ A′ B′	BC → CB′
⇒	1 0 1 D B′ A′ B′	1C → 1D
⇒	1 0 1 1 D A′ B′	DB′ → 1D
⇒	1 0 1 1 0 D B′	DA′ → 0D

Otro ejemplo: $w = 101$

PASO 14 / 15



regla `DB′ → 1D`



Otro ejemplo: $w = 101$

PASO 15 / 15

1 0 1 1 0 1

regla `D → ε`

> 101101 = (101)(101) ✅

⋮ 6 pasos anteriores

⇒	1 0 B 1 A B C	A1 → 1A
⇒	1 0 1 B A B C	B1 → 1B
⇒	1 0 1 B A C B′	BC → CB′
⇒	1 0 1 B C A′ B′	AC → CA′
⇒	1 0 1 C B′ A′ B′	BC → CB′
⇒	1 0 1 D B′ A′ B′	1C → 1D
⇒	1 0 1 1 D A′ B′	DB′ → 1D
⇒	1 0 1 1 0 D B′	DA′ → 0D
⇒	1 0 1 1 0 1 D	DB′ → 1D
⇒	**1 0 1 1 0 1**	D → ε

¿Y si C se adelanta? Un camino que se bloquea

PASO 0 / 9

S

símbolo inicial

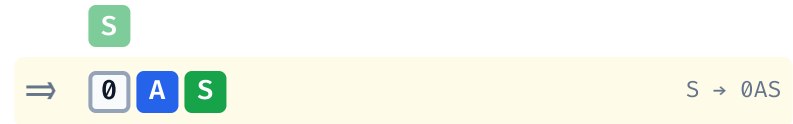
S

¿Y si C se adelanta? Un camino que se bloquea

PASO 1 / 9



regla `S → 0AS`

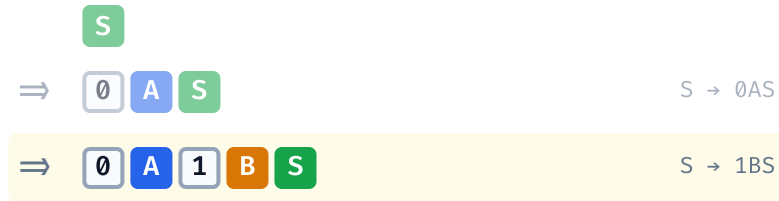


¿Y si C se adelanta? Un camino que se bloquea

PASO 2 / 9



regla S → 1BS



¿Y si C se adelanta? Un camino que se bloquea

PASO 3 / 9



regla S → C

S

⇒ 0 A S S → 0AS

⇒ 0 A 1 B S S → 1BS

⇒ 0 A 1 B C S → C

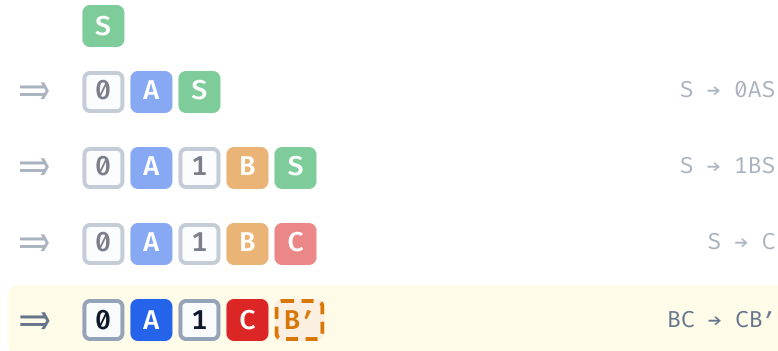
¿Y si C se adelanta? Un camino que se bloquea

PASO 4 / 9



regla `BC → CB′`

> C cruza B… pero A todavía no ha llegado.



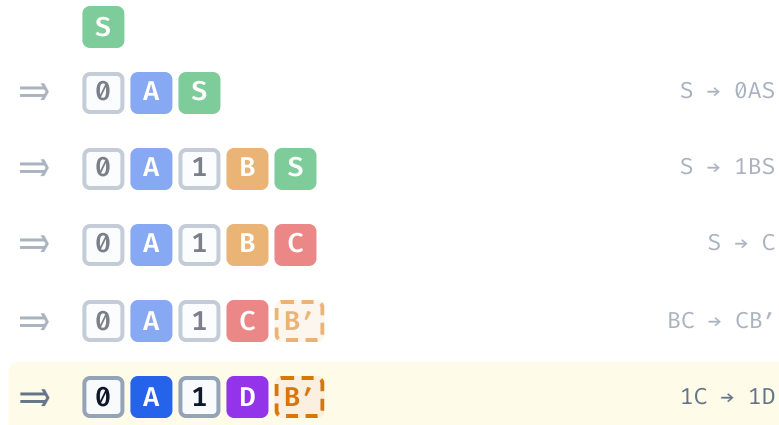
¿Y si C se adelanta? Un camino que se bloquea

PASO 5 / 9

0 A 1 D B′

regla 1C → 1D

> Mala elección: C toca el 1 y se convierte en D antes de visitar A.

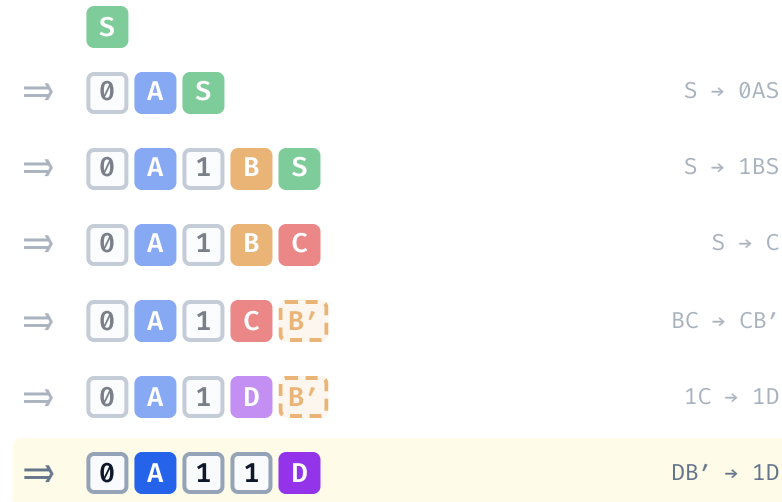


¿Y si C se adelanta? Un camino que se bloquea

PASO 6 / 9

0 A 1 1 D

regla DB′ → 1D

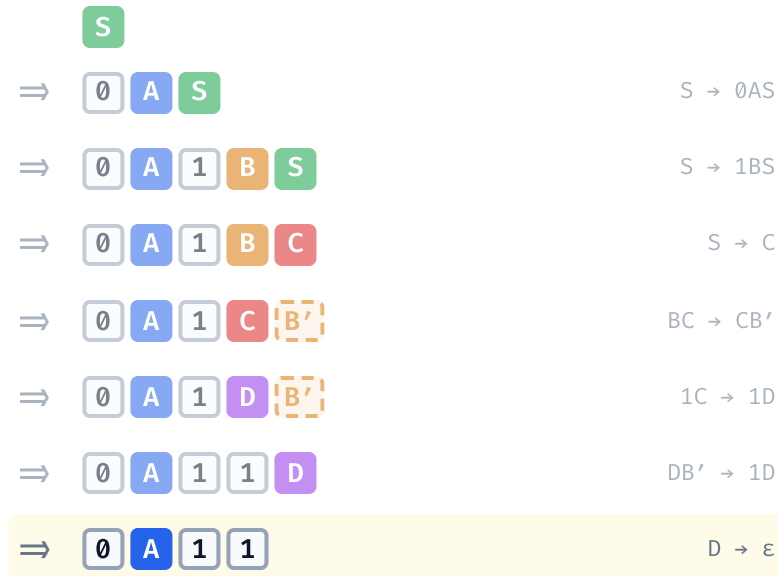


¿Y si C se adelanta? Un camino que se bloquea

PASO 7 / 9

0 A 1 1

regla `D → ε`



¿Y si C se adelanta? Un camino que se bloquea

PASO 8 / 9

0 1 A 1

regla A1 → 1A

S

⇒ 0 A S S → 0AS

⇒ 0 A 1 B S S → 1BS

⇒ 0 A 1 B C S → C

⇒ 0 A 1 C B′ BC → CB′

⇒ 0 A 1 D B′ 1C → 1D

⇒ 0 A 1 1 D DB′ → 1D

⇒ 0 A 1 1 D → ε

⇒ 0 1 A 1 A1 → 1A

¿Y si C se adelanta? Un camino que se bloquea

PASO 9 / 9

0 1 1 A

regla `A1 → 1A`

> ❌ Bloqueado: no hay ninguna regla que elimine A. Si pudiéramos borrarla saldría 011 ∉ L.



¿Por qué sólo genera palabras ww?

¿Por qué sólo genera palabras ww?

1. Los terminales que escribe S forman w y **ninguna regla los reordena**.

¿Por qué sólo genera palabras ww?

1. Los terminales que escribe S forman w y **ninguna regla los reordena**.
2. Los marcadores **nunca se adelantan entre sí** → siempre aparecen en el orden de w.

¿Por qué sólo genera palabras ww?

1. Los terminales que escribe S forman w y **ninguna regla los reordena**.
2. Los marcadores **nunca se adelantan entre sí** → siempre aparecen en el orden de w.
3. La **única** forma de eliminar un A o B es que C lo cruce ($AC \to CA'$).

¿Por qué sólo genera palabras ww?

1. Los terminales que escribe S forman w y **ninguna regla los reordena**.
2. Los marcadores **nunca se adelantan entre sí** → siempre aparecen en el orden de w.
3. La **única** forma de eliminar un A o B es que C lo cruce ($AC \to CA'$).
4. C sólo cruza marcadores; cuando toca un terminal pasa a ser D y **ya no puede volver**. → si quedaba algún marcador a su izquierda, se queda **bloqueado para siempre**.

¿Por qué sólo genera palabras ww?

1. Los terminales que escribe S forman w y **ninguna regla los reordena**.
2. Los marcadores **nunca se adelantan entre sí** → siempre aparecen en el orden de w.
3. La **única** forma de eliminar un A o B es que C lo cruce ($AC \to CA'$).
4. C sólo cruza marcadores; cuando toca un terminal pasa a ser D y **ya no puede volver**. → si quedaba algún marcador a su izquierda, se queda **bloqueado para siempre**.
5. Por tanto, en una derivación que termina, justo antes de crear D la forma es $w\,C$ (w codificada), p. ej.



¿Por qué sólo genera palabras ww?

1. Los terminales que escribe S forman w y **ninguna regla los reordena**.
2. Los marcadores **nunca se adelantan entre sí** → siempre aparecen en el orden de w.
3. La **única** forma de eliminar un A o B es que C lo cruce ($AC \to CA'$).
4. C sólo cruza marcadores; cuando toca un terminal pasa a ser D y **ya no puede volver**. → si quedaba algún marcador a su izquierda, se queda **bloqueado para siempre**.
5. Por tanto, en una derivación que termina, justo antes de crear D la forma es $w\,C$ (w codificada), p. ej. 

6. D escribe los marcadores de izquierda a derecha → resultado $w\,w$. ✅

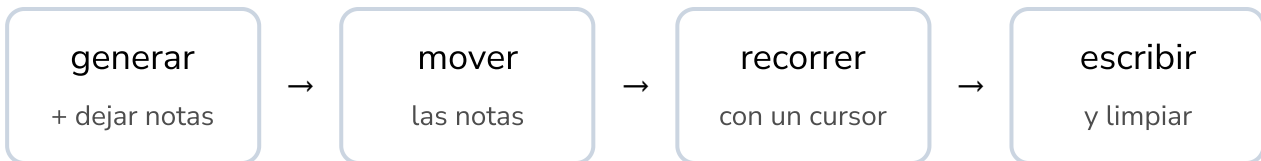
¿Por qué sólo genera palabras ww?

1. Los terminales que escribe S forman w y **ninguna regla los reordena**.
2. Los marcadores **nunca se adelantan entre sí** → siempre aparecen en el orden de w.
3. La **única** forma de eliminar un A o B es que C lo cruce ($AC \to CA'$).
4. C sólo cruza marcadores; cuando toca un terminal pasa a ser D y **ya no puede volver**. → si quedaba algún marcador a su izquierda, se queda **bloqueado para siempre**.
5. Por tanto, en una derivación que termina, justo antes de crear D la forma es $w\,C$ (w codificada), p. ej. 

6. D escribe los marcadores de izquierda a derecha → resultado $w\,w$. ✅

> Y al revés: para cualquier w hay derivación (mover todos los marcadores primero, luego C, luego D). Así que $L(G) = \{ww\}$.

Resumen del patrón



Este mismo esquema (marcadores que viajan + cursor que va y vuelve) lo usaremos en $\{a^{n^2}\}$.